NEXUS PROGRAMMING LANGUAGE

Next-Generation High-Convenience Automation & AI Language Specification

Version 1.0.0 (Automation Era) | Author: Antigravity AI Engine

1. Executive Summary & Design Philosophy

Nexus is designed to solve the critical friction points of modern programming languages. Current era languages force developers into compromises: Python offers readability but lacks speed and native process control; Rust provides memory safety and concurrency but imposes high compiler cognitive overhead; JavaScript offers ecosystem agility but suffers from async function coloring and callback fragility; Bash handles shell piping well but degrades quickly on structured data.

Why Nexus is 2X Advanced & Convenient:

- **1. Colorless Concurrency:** Lightweight green fibers eliminate function coloring (`async/await` overhead). Native `parallel { ... }` blocks run task branches concurrently with structured safety.
- **2. First-Class Shell & Stream Piping:** Shell commands (e.g. `$ dir /b`) are native syntax literals returning structured ProcessResult objects. The pipe operator (|>) flows data seamlessly between commands, arrays, and functions.
- **3. Native PC & System Automation:** Out-of-the-box standard library for hardware monitoring (CPU, RAM, Disks), OS process management, keyboard/mouse GUI automation, desktop notifications, and clipboard access.
- **4. Integrated Autonomous AI Agent Engine:** Built-in prompt evaluation (ai.prompt), schema extraction (ai.extract_json), and classification (ai.classify) directly inside language primitives.
- **5. Gradual Typing & Zero-Config Execution:** Write zero-boilerplate scripts instantly with type inference, while supporting optional sound static type annotations (`fn add(a: int, b: int) -> int`).

2. Language Syntax & Core Grammar

Nexus combines Pythonic clean syntax with C/Rust-style brace scoping. Expressions support string interpolation, lambda functions, process literals, and pipeline operators.

```
// Variable Declarations & Gradual Typing
let username = "Developer";           // Inferred String
const MAX_RETRY: int = 5;             // Type Annotated Constant

// String Interpolation with Expression Evaluation
let message = "User: {username}, Retries Left: {MAX_RETRY - 1}";

// Functions & Lambda Shorthand
fn calculate_tax(amount, rate: float) -> float {
    return amount * rate;
}
let numbers = [10, 20, 30, 40];
let doubled = numbers.map(x -> x * 2);
let filtered = numbers.filter(x -> x > 15);
```

3. Native Process Execution & Colorless Concurrency

In Nexus, process execution is elevated to a first-class language feature. Commands prefixed with $ or enclosed in backticks run OS processes natively and return ProcessResult strings equipped with helper methods.

```
// Native Process Execution & Stream Piping
let active_files = $ dir /b
                   |> lines()
                   |> filter(f -> f.contains(".nx") or f.contains(".py"));

// Colorless Concurrency: Parallel Block Execution
parallel {
    os.notify("Task A", "System audit starting...");
    fs.write("./backup.log", "Backup initiated at runtime");
    let net_info = http.get("https://httpbin.org/get");
}
```

4. Standard Library Automation Reference

Nexus includes built-in standard library modules covering 100% of PC and OS automation needs without external third-party dependencies.

Module	Function / Method Signature	Description & Return Type
os	`os.system_info() -> Map`	Returns CPU usage %, RAM total/used/free, Disk stats, and OS platform info.
os	`os.exec(cmd: str) -> Map`	Executes shell command returning {stdout, stderr, exit_code, success}.
os	`os.processes() -> Array[Map]`	Gets list of running PC processes with PID, CPU %, and Memory %.
os	`os.kill_process(target: str|int) -> Bool`	Terminates process by PID or matching process name.
os	`os.clip_read() -> String`	Reads text content from Windows / OS clipboard.
os	`os.clip_write(text: str) -> Bool`	Copies text string to Windows / OS clipboard.
os	`os.notify(title, message) -> Void`	Triggers OS desktop toast notification dialog.
fs	`fs.read(path: str) -> String`	Reads text file content from disk with UTF-8 encoding.
fs	`fs.write(path, content) -> Bool`	Creates directory path if needed and writes file content.
fs	`fs.list_dir(path: str) -> Array[String]`	Returns array of file/folder names inside directory.
fs	`fs.copy(src, dst) -> Bool`	Copies single file or entire directory recursively.
fs	`fs.remove(path: str) -> Bool`	Deletes target file or folder recursively.
fs	`fs.find_files(pattern) -> Array[String]`	Glob searches files matching pattern.
http	`http.get(url, headers) -> Map`	Performs HTTP GET request returning {status_code, body, json, ok}.
http	`http.post(url, body, json) -> Map`	Performs HTTP POST request with JSON payload or raw body.
http	`http.download(url, dest_path) -> Bool`	Downloads remote file stream directly to disk.
gui	`gui.click(x, y) -> Void`	Simulates left mouse button click at screen coordinate (x, y).
gui	`gui.move_mouse(x, y) -> Void`	Moves mouse cursor to absolute screen coordinate.
gui	`gui.type_text(text: str) -> Void`	Simulates typing text key sequence into focused window.
gui	`gui.alert(message, title) -> Void`	Displays modal GUI alert box to user.
ai	`ai.prompt(text: str) -> String`	Evaluates LLM / AI prompt returning structured answer.

ai	`ai.extract_json(text: str) -> Map`	Extracts and parses JSON block from unstructured text.
ai	`ai.classify(text, categories) -> String`	Classifies input text into one of the target categories.
data	`data.parse_json(str) / data.to_json(val)`	Parses and formats JSON data structures.
data	`data.sha256(text) / base64_encode(text)`	Computes SHA256 hashes and Base64 strings.
scheduler	`scheduler.sleep(seconds: float)`	Pauses current task execution thread.
scheduler	`scheduler.run_later(seconds, fn)`	Schedules background callback function execution.

5. End-to-End PC Automation Autobot Script

```
// 05_full_pc_autobot.nx — Complete PC Automation Script

fn pc_autobot() {
    print("■ NEXUS PC AUTOMATION BOT STARTING");

    // Step 1: PC System Metrics Audit
    let stats = os.system_info();
    print("RAM Used: {stats.ram_percent}%, CPU Used: {stats.cpu_usage_percent}%");

    // Step 2: File Directory Diagnostics
    let temp_files = fs.list_dir(".");
    print("Inspected workspace directory ({temp_files.length} items found).");

    // Step 3: Network Check & Report File Creation
    let http_check = http.get("https://httpbin.org/get");
    let report = {
        "cpu": stats.cpu_usage_percent,
        "ram": stats.ram_percent,
        "network_online": http_check.ok
    };
    fs.write("./pc_autobot_report.json", data.to_json(report));

    // Step 4: Clipboard Copy & Desktop Notification
    os.clip_write("PC Diagnostic: RAM {stats.ram_percent}%");
    os.notify("Nexus Autobot", "Full PC Diagnostic Completed!");
}

pc_autobot();
```

6. Nexus CLI Executable & REPL Toolchain

Nexus comes with a unified Command Line Interface toolchain supporting file execution, REPL shell interaction, and direct evaluation:

- **Run Script File:** python -m src.cli run script.nx
- **Interactive REPL:** python -m src.cli repl
- **Inline Evaluation:** python -m src.cli eval "let info = os.system_info(); print(info.ram_percent);"
- **System Info:** python -m src.cli info